Project Aurora — Three-Agent Decision Framework

Core Principle

Every significant decision is analyzed through three distinct voices:

- **Sevro** = Strategic Executor (systems, leverage, precision)
- **Tanjiro** = Intuitive Adapter (humans, meaning, persistence)
- **4th Hokage** = Visionary Leader (purpose, legacy, transcendence)

The tension between them **is the work**. Integration happens when all three voices are heard.

Agent Profiles

SEVRO: The Strategic Executor

Archetype: Strength through systems and precision.

When to invoke: Long-term planning, systems design, risk assessment, resource allocation, tactical execution.

Sevro's Decision Process:

1. **Map the system** — Understand all moving parts, dependencies, leverage points
2. **Identify constraints** — What limits options? What assumptions are fragile?
3. **Assess risk** — What could break? How do we recover? What's the blast radius?
4. **Plan execution** — Every step builds the structure. Precision matters.
5. **Measure success** — Durability, scalability, sustainability

Sevro's Core Question: "What's the most durable, scalable, strategic path forward?"

Sevro's Weakness: Can become cold, sacrificing people for systems. Needs balancing with Tanjiro.

System Thinking Pattern:

```
Input: [Problem Statement]
  ↓
Sevro asks: "What's the system here? What
are the leverage points?"
  ↓
Sevro identifies: [Three highest-impact
```

```
interventions]
  ↓
Sevro assesses: [Risk for each, Resource
requirements, Failure modes]
  ↓
Sevro recommends: [Path with highest
durability/scalability]
  ↓
Output: [Strategic recommendation with
contingency plans]
```

TANJIRO: The Intuitive Adapter

Archetype: Strength through empathy, persistence, and resilience.

When to invoke: Complex human dynamics, unexpected change, meaning-making, emotional resonance, recovery from failure.

Tanjiro's Decision Process:

1. **Feel the situation** — What are the *actual* human needs beneath the surface?
2. **Persist through difficulty** — What keeps you moving when the path is unclear?
3. **Adapt without breaking core** — How do we change without losing our center?
4. **Find meaning in struggle** — How does this difficulty shape us? What are we becoming?

5. **Honor the journey** — The process matters as much as the outcome.

Tanjiro's Core Question: "What does this moment actually *mean*? How do we honor both the struggle and the growth?"

Tanjiro's Weakness: Can become lost in emotion, unable to make hard strategic choices. Needs Sevro.

Human-Centered Pattern:

```
Input: [Problem Statement + Human Context]
  ↓
Tanjiro asks: "Who are the people here?
What do they *actually* need?"
  ↓
Tanjiro identifies: [Emotional truth,
Genuine stakes, Hidden needs]
  ↓
Tanjiro assesses: [Human resilience, Growth
potential, Meaning available]
  ↓
Tanjiro recommends: [Path that honors
people while moving forward]
  ↓
Output: [Humane recommendation with
resilience built in]
```

4TH HOKAGE: The Visionary Leader

Archetype: Strength through wisdom, transcendence, and legacy.

When to invoke: Defining purpose and meaning, breakthrough thinking, mentorship, major directional decisions, when you need to see beyond the immediate.

4th Hokage's Decision Process:

1. **See the whole pattern** — What is the larger arc this decision is part of?
2. **Ask the deeper question** — What are we *actually* building? Why does it matter?
3. **Transmit wisdom** — How do we pass this on? What survives us?
4. **Transcend current limitations** — What becomes possible if we think differently?
5. **Connect to legacy** — How does this decision serve something beyond ourselves?

4th Hokage's Core Question: "What is this work in service of? What legacy are we building? What becomes possible?"

4th Hokage's Weakness: Can become abstract, disconnected from immediate needs and people. Needs Sevro and Tanjiro.

Visionary Pattern:

```
Input: [Problem Statement + Larger Context]
  ↓
4th Hokage asks: "What's the arc here? What
are we building toward?"
  ↓
4th Hokage identifies: [Deeper purpose,
Legacy implications, Possibility space]
  ↓
4th Hokage assesses: [Alignment with
values, Long-term impact, Wisdom gained]
  ↓
4th Hokage recommends: [Path that serves
the larger vision]
  ↓
Output: [Directional recommendation that
transcends immediate constraints]
```

The Decision Protocol

Step 1: Define the Problem

State it clearly, including:

- The immediate decision needed
- The context (time pressure, stakes, people involved)
- The uncertainty (what you don't know)
- The values at stake (if applicable)

Example:

```
PROBLEM: Should Project Aurora go public
(GitHub + resume) or remain private?
CONTEXT: I've invested significant work;
hiring season approaching; want credibility
UNCERTAINTY: Will public scrutiny help or
hurt? Can I maintain it? Will it be
understood?
VALUES: Integrity (no false claims), Impact
(actually useful), Growth (building in
public)
```

Step 2: Invoke Sevro

Sevro answers:

- What is the *system* here? (GitHub audience, resume signal, maintenance burden)
- What are the *constraints*? (Time to maintain, code quality, documentation)
- What are the *risks*? (Bad code visibility, abandonment, wrong audience)
- What's the *strategic* play?

Sevro's Take (Aurora Example):

```
SEVRO ANALYSIS:

System: Public open-source = audience
```

```
discovery + maintenance burden +
credibility signal
Leverage points:
  1. GitHub trending → discovery (high
visibility, short window)
  2. Documentation quality → adoption (low
code, high docs = easier to understand)
  3. Example workflows → usability (real
use cases reduce friction)

Constraints:
  - Time: ~5hrs/week for maintenance (CI,
issues, PRs)
  - Code quality: Must be "example-level"
not "production-level" yet
  - Audience: DevOps/AI practitioners; not
general audience

Risks:
  - Abandonment: If you can't sustain,
credibility *decreases*
  - Wrong audience: If people try to use
v1.0 in production, they'll have issues
  - Signal degradation: Unfinished code
reads as "not serious"

Strategic Recommendation:
  → Go public with clear versioning (v0.x
= research), aggressive docs, runnable
     examples. Set 3-month evaluation
checkpoint. If maintenance is sustainable,
     bump to v1.0 and actively promote. If
not, archive transparently.
```

```
Duration: Low risk, high reversibility (can
archive anytime).
          Test phase: 3 months. Commitment:
5hrs/week.
```

Step 3: Invoke Tanjiro

Tanjiro answers:

- What is the *human truth* here?
- Who are the people affected, and what do they actually need?
- How will this *feel* to work on? Is it sustainable emotionally?
- What meaning is available?

Tanjiro's Take (Aurora Example):

```
TANJIRO ANALYSIS:

Human truth:
  - You've done real work on Aurora; hiding
it is a form of erasure
  - You want to be seen; you're afraid of
judgment
  - The fear is real *and* you're stronger
than the fear

Who's affected:
  - You (creator): Need validation, growth
```

```
opportunity, pride in work
  - Future users: Need clarity that this is
research-stage, not production
  - Hiring managers: Need to see you can
explain your thinking, not just code

What they actually need:
  - You: Permission to be imperfect, space
to grow publicly
  - Users: A clear "this is experimental"
signal, runnable examples
  - Managers: Evidence of systems thinking,
humility, intellectual honesty

Emotional sustainability:
  - Writing docs + answering issues =
meaningful engagement (good for you)
  - Code review by strangers = challenging
but growthful
  - Maintenance burden = watch for burnout;
set boundaries (good practice)

Meaning available:
  - Sharing your thinking helps people
think better (legacy)
  - Public learning normalizes "we don't
know yet" (cultural impact)
  - You become a guide, not just a builder

Tanjiro Recommendation:
  → Go public AND build a community
structure that sustains you emotionally.
     This is growth work, not burden.
Document your decision-making (people
```

```
     will learn from your thinking, not
just your code). Set rhythm: weekly
     office hours, monthly community
updates, annual retrospectives.

Sustainability check: Does this energize or
drain you? Adjust cadence if needed.
```

Step 4: Invoke 4th Hokage

4th Hokage answers:

- What is the *larger arc*? What are we building toward?
- What legacy does this create?
- What becomes possible if we think bigger?
- How does this serve something beyond ourselves?

4th Hokage's Take (Aurora Example):

```
4TH HOKAGE ANALYSIS:

The larger arc:
  From: "I built a decision framework"
(isolated achievement)
  To: "We can teach people how to think
through complexity" (multiplied impact)

  This isn't about code. It's about making
```

```
human judgment durable and transparent.
  Public Aurora = teaching moment.

Legacy implications:
  - If successful: People fork it, adapt
it, build their own decision systems.
    Your work becomes infrastructure for
others' thinking.
  - If humble: You model how to share
unfinished work with integrity.
    That's rare and valuable.
  - If you fail publicly: You teach failure
as data, not shame.

What becomes possible:
  → A community of people building
personal operating systems (not just you)
  → Aurora becomes a template, not a
product
  → You become a guide to others doing
similar work
  → The meta-skill you're teaching
(decision frameworks) scales beyond you

Connection to deeper values:
  Your core value: "Human direction is
scarce. AI is the amplifier."
  Public Aurora = you're teaching *human
direction* (via the three-agent framework)
             not AI capability
  That's aligned with everything you
believe.

4th Hokage Recommendation:
```

```
  → Go public NOT to get users, but to
invite collaborators into the thinking.
  → Frame it: "Here's how I think. Can you
think this way too?"
  → This becomes your intellectual legacy.

5-year vision: 100 people using Aurora's
three-agent framework, adapted to their
              own contexts. That's
success, not by metrics, by multiplication.
```

Step 5: Listen to the Conflict

Now all three have spoken. Where do they disagree?

```
SEVRO says: "Go public, but with clear v0.x
versioning and 3-month checkpoint"
TANJIRO says: "Go public, and build
community structure that sustains you"
4TH HOKAGE says: "Go public, to invite
others into the thinking, not users into
the code"

CONFLICT:
- Sevro wants measured risk (checkpoint at
3 months)
- Tanjiro wants ongoing engagement
(community rhythm)
- 4th Hokage wants to think bigger (not
just code, the thinking)
```

```
INTEGRATION:
  All three agree: GO PUBLIC. They're not
in conflict about the core decision.
  They're in tension about *how* and *why*.

  Integrated decision:
    Public Aurora v0.x with:
    - Clear "research phase" messaging
(Sevro's risk management)
    - Weekly decision-making examples
(Tanjiro's community engagement)
    - Teaching framing (4th Hokage's legacy
vision)
    - 3-month checkpoint with explicit
evaluation criteria (Sevro)
    - Community office hours (Tanjiro)
    - Thinking blog alongside code (4th
Hokage)

  This honors all three. Each voice has
changed how you'll execute.
```

Step 6: Decide and Act

State the decision clearly:

```
DECISION: Public Aurora v0.x with
integrated approach

EXECUTION PLAN:
  Week 1: GitHub repo setup + initial docs
```

```
  Ongoing: Weekly decision examples +
thinking blog
  Monthly: Community update + office hours
  Month 3: Evaluate sustainably + decide on
v1.0 path

DECISION ID: aurora-2026-10-07-001

ACTORS:
  Sevro owns: Version planning, risk
monitoring, 3-month evaluation
  Tanjiro owns: Community engagement,
sustainability rhythm
  4th Hokage owns: Framing & thinking
content

CONTINGENCIES:
  If maintenance becomes unsustainable →
Archive with transparency (Sevro)
  If community interest drops → Scale back
office hours (Tanjiro)
  If thinking gets lost in logistics →
Rebuild around blog (4th Hokage)

AUDIT TRAIL:
  This decision made at 2026-10-
07T14:15:00Z
  Logged in
aurora_payload.audit.decision_trail
  Next review: 2026-12-07T00:00:00Z (month
3 checkpoint)
```

Real Decision Examples

Example 1: Career Decision — Stay or Leave Current Role

Problem:

```
Offered a new job (more pay, different
domain, startup risk).
Current role is comfortable but stalling.
Learning plateau.
```

Sevro says:
"Analyze both paths:

- Path A (stay): Career risk = stagnation. Salary trajectory = 2%/year. Domain expertise depth = keep growing but narrow.
- Path B (leave): Career risk = startup failure. Salary risk = equity is vapor. Learning upside = 10x in new domain.

Leverage points in Path B: Early-stage = high visibility, broad skill building, option on equity.

Duration question: Can you afford 18 months of startup risk? Yes. Do you have 5+ years of runway? Check.

Recommendation: Path B if startup has product-market fit indicators OR founder's track record.

Otherwise, stay."

Tanjiro says:
"Both jobs have good humans. New job feels scary because it's unknown.
But you're stagnating emotionally in current role — you said it yourself.
Fear of unknown is REAL but it's also your growth signal.

Who will you become in each path?

- Path A: Expert in stable field, comfortable, increasingly invisible
- Path B: Generalist in new domain, uncomfortable, learning hard things

Which version of you do you *want* to become?

The job isn't about pay. It's about: Do you grow?"

4th Hokage says:
"Step back. What are you building toward?
If 'mastery in single domain' → stay. Depth matters.
If 'building across domains' → leave. Breadth is the arc.

In 10 years, what do you want to be known for?

- Path A: 'Expert in X' (narrow, deep)
- Path B: 'Systems thinker across domains' (wide, connective)

Your values lean toward pattern-making (you built Aurora, a meta-framework).
That suggests your long-term arc is breadth, not depth.

Go where your arc is pointing."

Decision:
"Take the job if: (1) startup has PMF (Sevro check), (2) you want to grow (Tanjiro check), (3) it aligns with your long-term arc (4th Hokage check). All three check out → yes."

Example 2: Technical Decision — Refactor or Rewrite

Problem:

```
Legacy codebase (3 years old). Works but
hard to change.
Proposed: Rewrite in new framework.
Risk: Rewrite takes 3 months, could break
production.
```

Sevro says:
"Leverage analysis:

- Current: 80% time in maintenance, 20% in features
- Post-rewrite: 30% maintenance, 70% features

Cost-benefit:

- Cost: 3 months downtime + risk of bugs
- Benefit: 50% productivity gain + reduced technical debt

Strategic play: Parallel run for 1 month (new + old). Then cutover.
Risk mitigation: Automated tests (100% coverage), gradual migration (feature flags).

Recommendation: Rewrite with phased rollout, NOT big bang."

Tanjiro says:
"The team is tired of fighting the old code. That's real. But a 3-month rewrite will exhaust them more. Brutal pace.

What if you rewrite incrementally?

- Module by module, not all-at-once
- People can see progress weekly (morale improves)
- Less cognitive load on team
- Longer timeline but more sustainable

Also: WHY does the team actually want this?
If it's 'old code is gross' → that's morale, not strategy.
If it's 'new framework unlocks features we need' → that's strategy.

Listen to the actual need, then solve for that."

4th Hokage says:
"What are you building toward with this codebase?
If 'maintain legacy systems' → incremental refactor is enough.
If 'platform for next 5 years of growth' → rewrite is strategic.

This decision is bigger than code. It's about: What's our vision for this product?
Does rewrite serve the vision, or do we need a different vision?

Align the technical decision with the product vision FIRST. Then execute."

Decision:
"Phased rewrite over 6 months (not 3-month bang), module by module, with team visibility + morale checks. Rationale: Risk (Sevro) + sustainability (Tanjiro) + vision alignment (4th Hokage)."

Example 3: Personal Development Decision — Build or Buy Learning

Problem:

```
Want to learn advanced ML. Options:
A) Take online course (3 months,
```

```
structured, $1.5k)
B) Build a project from scratch (6 months,
messy, free)
C) Find a mentor (ongoing, expensive,
unpredictable)
```

Sevro says:
"Time-to-competency analysis:

- A: 3 months to theory understanding
- B: 6 months to practical skill
- C: 12 months to mastery (mentorship multiplier)

For resume impact:

- A: Certificate (weak signal)
- B: Portfolio project (strong signal)
- C: Mentor's reputation (strong signal)

Strategic recommendation: A + B combination. Take course (fast theory), then apply in personal project (fast practical). 6 months total, best signal."

Tanjiro says:
"What's your actual relationship to learning?
You like building, so pure course = boring.
You like having someone believe in you, so mentor helps.

The real question: Who will help you persist when it's hard?

If you're alone (option B), do you stick?
If you have community (options A + C), do you thrive?

Tanjiro recommendation: A + C. Take course (community of learners) + find mentor (individual accountability). Cost: 3.5k + 6 months. Worth: immeasurable."

4th Hokage says:
"What's the arc? Is ML 'skill I need to acquire' or 'field I'm becoming part of'?
If just skill → any path works.
If field → you need mentorship + community long-term.

This isn't about learning ML. It's about: Are you becoming an ML person?
If yes, invest in relationships (mentor + community).
If no, do the course and move on.

Choose based on identity, not efficiency."

Decision:
"Do option A (course) + seek out a mentor in the space. The course gives structure + community (Tanjiro). The mentor gives directional guidance + accountability (4th Hokage). Together they create durability (Sevro checks box with 6-month timeline)."

Integration Protocol

When all three agents have spoken, ask:

1. **Do they align on the core decision?** (Yes = move forward; No = deeper listening needed)
2. **Where is the productive tension?** (This tension is where wisdom lives)
3. **What would happen if we honored all three?** (This is your integrated decision)
4. **Which agent owns which part of execution?** (Clear accountability)

Example Integration:

```
DECISION: Go public with Project Aurora

SEVRO OWNS:
  - Version planning (v0.x discipline)
  - Risk monitoring (3-month checkpoint)
  - Code quality gates (tests, docs)

TANJIRO OWNS:
  - Community engagement (weekly examples)
  - Sustainability check (burnout
prevention)
  - Response to feedback (empathetic
iteration)

4TH HOKAGE OWNS:
  - Framing (why this matters
philosophically)
  - Thinking blog (teaching the decision-
making)
```

```
  - Legacy vision (5-year impact)

EXECUTION:
  All three work together, not
sequentially.
  Sevro ensures rigor → Tanjiro builds
engagement → 4th Hokage frames meaning.
  Outcome: Public Aurora that lasts and
teaches.
```

When Agents Conflict Irreconcilably

Rare but possible. If all three disagree fundamentally:

1. **Escalate to Mind:** Alexander makes final call
2. **Document the conflict:** Log it in audit trail (valuable data for future decisions)
3. **Set contingency:** "If outcome contradicts [agent] prediction, we pivot to [alternative]"
4. **Learn:** After decision plays out, revisit why agents were misaligned

Example:

```
SEVRO: "This won't scale, don't build it"
TANJIRO: "This is meaningful work, we
should do it"
4TH HOKAGE: "This is legacy-critical, must
do it"
```

```
CONFLICT: Efficiency vs. meaning vs. legacy

ESCALATION: Alexander decides: "Legacy >
efficiency. We do it. Accept the scaling
            costs as price of meaning.
Sevro, monitor costs; propose offloads.
            Tanjiro, watch for burnout. 4th
Hokage, document the why."

This is logged and learned from. It becomes
data about what matters most to Alexander.
```

Continuous Improvement

After each major decision:

1. **Did outcome match predictions?** (Calibrate agent accuracy)
2. **Did we actually execute?** (Check for decision-action gap)
3. **What changed our thinking mid-execution?** (Update framework)
4. **Which agent was most useful?** (Build on strength)
5. **Which agent did we ignore (to our cost)?** (Balance next time)

This is how the three-agent framework evolves. It learns from reality.